UNIT-I:
Introduction to Machine Learning and Pre-requisites: Introduction to Machine Learning – Learning Paradigms – PAC learning – Version Spaces – Role of Machine Learning in Artificial Intelligence applications.

Introduction

Definition of machine learning

Arthur Samuel, an early American leader in the field of computer gaming and artificial intelligence, coined the term "Machine Learning" in 1959 while at IBM. He defined machine learning as "the field of study that gives computers the ability to learn without being explicitly programmed." However, there is no universally accepted definition for machine learning.

Different authors define the term differently. We give below two more definitions.

- Machine learning is programming computers to optimize a performance criterion using example data or past experience. We have a model defined up to some parameters, and learning is the execution of a computer program to optimize the parameters of the model using the training data or past experience. The model may be predictive to make predictions in the future, or descriptive to gain knowledge from data, or both.
- The field of study known as machine learning is concerned with the question of how to construct computer programs that automatically improve with experience .

Definition of Machine learning

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks T, as measured by P, improves with experience E.

Examples

i) Handwriting recognition learning problem
- Task T: Recognising and classifying handwritten words within images
- Performance P: Percent of words correctly classified
- Training experience E: A dataset of handwritten words with given classifications

ii) A robot driving learning problem
- Task T: Driving on highways using vision sensors
- Performance measure P: Average distance traveled before an error
- training experience: A sequence of images and steering commands recorded while observing a human driver

iii) A chess learning problem
- Task T: Playing chess
- Performance measure P: Percent of games won against opponents
- Training experience E: Playing practice games against itself

Definition

A computer program which learns from experience is called a machine learning program or simply a learning program. Such a program is sometimes also referred to as a learner.

Understanding data

The different types and forms of data that are encountered in the machine learning process.

Unit of observation

By a unit of observation we mean the smallest entity with measured properties of interest for a study.

Examples

- A person, an object or a thing
- A time point
- A geographic region
- A measurement Sometimes, units of observation are combined to form units such as person-years.

Examples and features

Datasets that store the units of observation and their properties can be imagined as collections of
data consisting of the following:

Examples

An "example" is an instance of the unit of observation for which properties have been recorded.

An "example" is also referred to as an "instance", or "case" or "record." (It may be noted that the word "example" has been used here in a technical sense.)

Features

A "feature" is a recorded property or a characteristic of examples. It is also referred to as "attribute", or "variable" or "feature."

features

year	model	price	mileage	color	transmission
2011	SEL	21992	7413	Yellow	AUTO
2011	SEL	20995	10926	Gray	AUTO
2011	SEL	19995	7351	Silver	AUTO
2011	SEL	17809	11613	Gray	AUTO
2012	SE	17500	8367	White	MANUAL
2010	SEL	17495	25125	Silver	AUTO
2011	SEL	17000	27393	Blue	AUTO
2010	SEL	16995	21026	Silver	AUTO
2011	SES	16995	32655	Silver	AUTO

examples

1. Cancer detection

Consider the problem of developing an algorithm for detecting cancer. In this study we note
the following.

(a) The units of observation are the patients.

(b) The examples are members of a sample of cancer patients.

(c) The following attributes of the patients may be chosen as the features:

- gender
- age
- blood pressure

- the findings of the pathology report after a biopsy

2. Pet selection
Suppose we want to predict the type of pet a person will choose.
(a) The units are the persons.
(b) The examples are members of a sample of persons who own pets. Figure 1.2: Example for "examples" and "features" collected in a matrix format (data relates toautomobiles and their features)
(c) The features might include age, home region, family income, etc. of persons who own pets.

3. Spam e-mail
Let it be required to build a learning algorithm to identify spam e-mail.
(a) The unit of observation could be an e-mail messages.
(b) The examples would be specific messages.
(c) The features might consist of the words used in the messages.
Examples and features are generally collected in a "matrix format". Fig. 1.2 shows such a data
set.

Different forms of data

- Numeric data
  If a feature represents a characteristic measured in numbers, it is called a numeric feature.
- Categorical or nominal
  A categorical feature is an attribute that can take on one of a limited, and usually fixed, number of possible values on the basis of some qualitative property. A categorical feature is also called a nominal feature.
- Ordinal data
  This denotes a nominal variable with categories falling in an ordered list. Examples include clothing sizes such as small, medium, and large, or a measurement of customer satisfaction on a scale from "not at all happy" to "very happy."

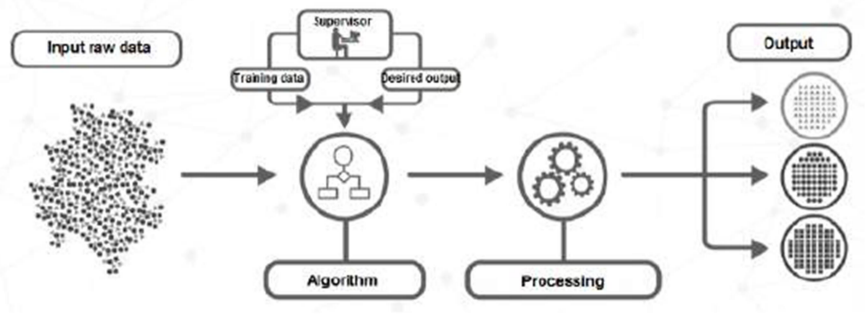
Examples
In the data given in Fig.1.2, the features "year", "price" and "mileage" are numeric and the features
"model", "color" and "transmission" are categorical.

Different types of learning Paradigms in Machine Learning
In general, machine learning algorithms can be classified into three types.

Supervised learning
Supervised learning is the machine learning task of learning a function that maps an input to an output based on example input-output pairs.

In supervised learning, each example in the training set is a pair consisting of an input object (typically a vector) and an output value. A supervised learning algorithm analyzes the training data and produces a function, which can be used for mapping new examples. In the optimal case, the function will correctly determine the class labels for unseen instances. Both classification and regression problems are supervised learning problems.

A wide range of supervised learning algorithms are available, each with its strengths and weaknesses. There is no single learning algorithm that works best on all supervised learning problems.



A "supervised learning" is so called because the process of algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. We know the correct answers (that is, the correct outputs), the algorithm iteratively makes predictions on the training data and
is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance.

Example
Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients and each patient is labeled as "healthy" or "sick".

gender	age	label
M	48	sick
M	67	sick
F	53	healthy
M	49	healthy
F	34	sick
M	21	healthy

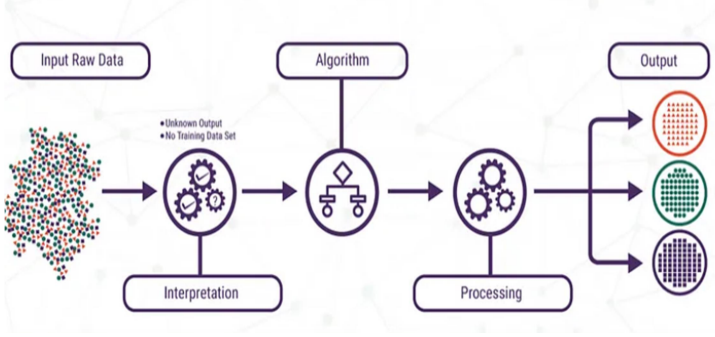
Based on this data, when a new patient enters the clinic, how can one predict whether he/she is healthy or sick?

Unsupervised learning

Unsupervised learning is a type of machine learning algorithm used to draw inferences from datasets consisting of input data without labeled responses.

In unsupervised learning algorithms, a classification or categorization is not included in the observations. There are no output values and so there is no estimation of functions. Since the examples given to the learner are unlabeled, the accuracy of the structure that is output by the algorithm cannot be evaluated.

The most common unsupervised learning method is cluster analysis, which is used for exploratory data analysis to find hidden patterns or grouping in data.



Example

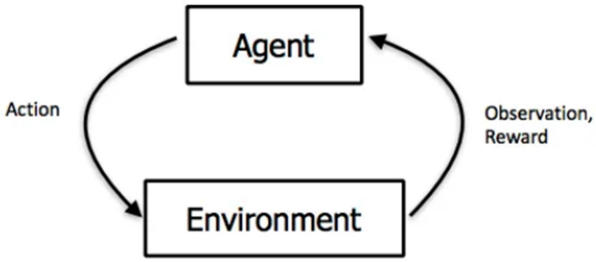
Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients.

gender	age
M	48
M	67
F	53
M	49
F	34
M	21

Based on this data, can we infer anything regarding the patients entering the clinic?

Reinforcement learning

Reinforcement Learning is a type of Machine Learning, and thereby also a branch of Artificial Intelligence. It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance.



Learning pattern in reinforcement learning

There is an excellent analogy to explain this type of learning paradigm, “training a dog”. This learning paradigm is like a dog trainer, which teaches the dog how to respond to specific signs, like a whistle, clap, or anything else. Whenever the dog responds correctly, the trainer gives a reward to the dog, which can be a “Bone or a biscuit”.

A variety of different problems can be solved using Reinforcement Learning. Because RL agents can learn without expert supervision, the type of problems that are best suited to RL are complex problems where there appears to be no obvious or easily programmable solution.

Two of the main ones are:

- Game playing — determining the best move to make in a game often depends on a number of different factors, hence the number of possible states that can exist in a particular game is usually very large.
- Control problems — such as elevator scheduling. Again, it is not obvious what strategies would provide the best, most timely elevator service. For control problems such as this, RL agents can be left to learn in a simulated environment and eventually they will come up with good controlling policies.

	Supervised Learning	Unsupervised Learning
Definition	It's a learning setup commonly used in machine learning where human supervision is involved as they label the data and provide target output to the algorithm to map the input to it	It's a learning setup commonly used in machine learning where human supervision is minimal, as the model finds the patterns in the data without any supervision
Input Data	Data has labels	Data doesn't have labels
Data Usage	The data has x features and Y variable, and the model finds Y = f(x)	Patterns are found in the x features of the data as no Y variable is present
When to use	Know the expected outcome and what is being looked for	Don't know the expected outcome and what is being looked for
Nature of Problems / Types	Regression and Classification	Clustering, Dimensionality Reduction, and Association
Goal	Predict outcomes for new data based on training data	Get hidden patterns and useful insights from large datasets
Output	Predicted labels	Clusters or association rules
Associated Algorithms	Linear Regression, Logistic Regression, SVM, KNN	K-means, DBSCAN, PCA, Apriori Algorithm
Proximity to AI	Farther from AI	Closer to AI as, human supervision in learning is relatively less
Complexity	Simple and requires less data	Complex and requires a lot of data
Drawbacks	Training is time-consuming, and it isn't easy to obtain labels	Human intervention is required to validate results and evaluate model performance. Inaccurate and unreliable results
Expected Accuracy	High	Low
Use Cases	Demand and weather forecasting, spam filters, image recognition, and price prediction	Anomaly detection, customer segmentation, recommender system, medical imaging, producing labels for performing supervised learning

Differences between Supervised ,unsupervised and Reinforcement Learning

Criteria	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Input Data	Input data is labelled.	Input data is not labelled.	Input data is not predefined.
Problem	Learn pattern of inputs and their labels.	Divide data into classes.	Find the best reward between a start and an end state.
Solution	Finds a mapping equation on input data and its labels.	Finds similar features in input data to classify it into classes.	Maximizes reward by assessing the results of state-action pairs
Model Building	Model is built and trained prior to testing.	Model is built and trained prior to testing.	The model is trained and tested simultaneously.
Applications	Deal with regression and classification problems.	Deals with clustering and associative rule mining problems.	Deals with exploration and exploitation problems.
Algorithms Used	Decision trees, linear regression, K-nearest neighbors	K-means clustering, k-medoids clustering, agglomerative clustering	Q-learning, SARSA, Deep Q Network
Examples	Image detection, Population growth prediction	Customer segmentation, feature elicitation, targeted marketing, etc	Drive-less cars, self-navigating vacuum cleaners, etc

Concept Learning and Version Space

We learn our surrounding through 5 senses — eye, ear, nose, tongue and skin. We learn lot of things during the entire life. Some of them are based on experience and some of them are based on memorization. On the basis of that we can divide learning methods into five types:

- Rote Learning (memorization): Memorizing things without knowing the concept/ logic behind them.
- Passive Learning (instructions): Learning from a teacher/expert.
- Analogy (experience): Learning new things from our past experience.
- Inductive Learning (experience): On the basis of past experience, formulating a generalized concept.
- Deductive Learning: Deriving new facts from past facts.

In machine learning, our interest is in inductive learning and its based on formulating a generalized concept after observing a number of instances of examples of the concept.

Assume that we have collected data for some attributes/features of the day like, Sky, Air Temperature, Humidity, Wind, Water, Forecast.

Let these set of instances be denoted by X and many concepts can defined over the X. For example, the concepts can be
- Days on which my friend Sachin enjoys his favorite water sport
- Days on which my friend Sachin will not go outside of his house.
- Days on which my friend Sachin will have night drink.

Target concept — The concept or function to be learned is called the target concept and denoted by c. It can be seen as a boolean valued function defined over X and can be represented as c: X → {0, 1}.
For the target concept c , "Days on which my friend Sachin enjoys his favorite water sport", an attribute EnjoySport is included in the below dataset X and it indicates whether or not my friend Sachin enjoys his favorite water sport on that day.

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Now the target concept is EnjoySport : X →{0,1}.
With this, a learner task is to learn to predict the value of EnjoySport for arbitrary day, based on the values of its attribute values.

When a new sample with the values for attributes <Sky, Air Temperature, Humidity, Wind, Water, Forecast> is given, the value for EnjoySport (ie. 0 or 1) is predicted based on the previous learning.

Let H denote the set of all possible hypotheses that the learner may consider regarding the identity of the target concept. So H = {h1, h2, …. }.
What hypothesis representation shall we provide to the learner in this case ?

Let us begin by considering a simple representation in which each hypothesis consists of a conjunction of constraints on the instance attributes.
Let each hypothesis be a vector of six constraints, specifying the values of the six attributes <Sky, Air Temperature, Humidity, Wind, Water, Forecast>.
In hypothesis representation, value of each attribute could be either

- "?' — that any value is acceptable for this attribute,
- specify a single required value (e.g., Warm) for the attribute, or
- "0" that no value is acceptable.

For example :
the hypothesis that my friend Sachin enjoys his favorite sport only on cold days with high humidity (independent of the values of the other attributes) is represented by the expression < ?,cold, High,?, ? ,?>

- The most general hypothesis — < ?, ? , ? , ?, ? , ?> that every day is a positive example
- The most specific hypothesis — < 0, 0, 0, 0, 0, 0>that no day is a positive example.

With this knowledge we can say, EnjoySport concept learning task requires

- learning the sets of days for which EnjoySport=yes and then
- describing this set by a conjunction of constraints over the instance attributes.

EnjoySport Concept Learning Task

Now lets define the learning task for EnjoySport Concept.
The below points covers the simplified overview.

- **X —** The set of items over which the concept is defined is called the set of instances, which we denote by X. In the current example, X is the set of all possible days, each represented by the attributes Sky, AirTemp, Humidity, Wind, Water, and Forecast.
- **c —** The concept or function to be learned is called the target concept, which we denote by c. In general, c can be any boolean valued function defined over the instances X; that is, c: X → {0, 1}. In the current example, the target concept corresponds to the value of the attribute EnjoySport (i.e, c(x)=1 if EnjoySport=Yes, and c(x)=0 if EnjoySport= No).
- **(x, c(x)) —** When learning the target concept, the learner is presented by a set of training examples, each consisting of an instance x from X, along with its target concept value c(x). Instances for which c(x) = 1 are called positive examples and instances for which c(x) = 0 are called negative examples. We will often write the ordered pair (x, c(x)) to describe the training example consisting of the instance x and its target concept value c(x).
- **D —** We use the symbol D to denote the set of available training examples.
- **H —** Given a set of training examples of the target concept c, the problem faced by the learner is to hypothesize, or estimate, c. We use the symbol H to denote the set of all possible hypotheses that the learner may consider regarding the identity of the target concept.
- **h(x) —** In general, each hypothesis h in H represents a Boolean-valued function defined over X; that is, h : X →{0, 1}. The goal of the learner is to find a hypothesis h such that h(x) = c(x) for all x in X.

Given

- ***Instances X*** : set of all possible days, each described by the attributes
  - Sky – (values: Sunny, Cloudy, Rainy)
  - AirTemp – (values: Warm, Cold)
  - Humidity – (values: Normal, High)
  - Wind – (values: Strong, Weak)
  - Water – (values: Warm, Cold)
  - Forecast – (values: Same, Change)
- ***Target Concept (Function) c*** : EnjoySport : X → {0,1}
- ***Hypotheses H*** : Each hypothesis is described by a conjunction of constraints on the attributes.
- ***Training Examples D*** : positive and negative examples of the target function

Determine

- A hypothesis h in H such that h(x) = c(x) for all x in D.

Notice that, the learning algorithm objective is to find a hypothesis h in H such that h(x) = c(x) for all x in D.
We know that, the Inductive learning algorithm tries to induce a "general rule" from a set of observed instances. So the above case is same as inductive learning where a learning algorithm is trying to find a hypothesis h (general rule) in H such that h(x) = c(x) for all x in D. For a given collection of examples, in reality, learning algorithm return a function h (hypothesis) that approximates c (target concept). But the expectation is, the learning algorithm to return a function h (hypothesis) that equals c (target concept) ie. h(x) = c(x) for all x in D.
So we can define, Inductive learning hypothesis is any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over any other unobserved examples.

Hypothesis Space

- Let X denote the instances and H as hypotheses in the EnjoySport learning task.
- Lets compute the distinct instances and hypothesis in X and H respectively as below.
- Out hypothesis h is a vector of six constraints, specifying the values of the six attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>**.
- In this hypothesis representation, value of each attribute could be either "?" or "0" other than defined values. So the hypothesis space H has 5120 distinct hypothesis.

Attributes	Values	Count
1. Sky	Sunny, Cloudy, Rainy	3
2. AirTemp	Warm, Cold	2
3. Humidity	Normal, High	2
4. Wind	Strong, Weak	2
5. Water	Warm, Cool	2
6. Forecast	Same, Change	2

$$Distinct\ Observations\ in\ X = 3.2.2.2.2.2 = 96$$

$$Distinct\ hypothesis\ in\ H = 5.4.4.4.4.4 = 5120$$

The number of combinations: 5×4×4×4×4×4 = 5120 syntactically distinct hypotheses. They are syntactically distinct but not semantically. For example, the below 2 hypothesis says the same but they look different.
h1 = <Sky=0 AND Temp=warm AND Humidity=? AND Wind=strong AND Water=warm AND Forecast=same >
h2 = <Sky=sunny AND Temp=warm AND Humidity=? AND Wind=strong AND Water=0 AND Forecast=same>
Neither of these hypotheses accept any "day", so semantically the same. All such hypothesis having same semantic is counted as 1. So we can have total number of combinations as below.
1 (hypothesis with one or more 0) + 4×3×3×3×3×3 (add ? to each attribute) = 973 semantically distinct hypotheses

We have already seen that, the concept learning can be viewed as the task of searching through a large space of hypotheses. The goal of this search is to find the hypothesis that best fits the training examples. By selecting a particular hypothesis representation, the designer of the learning algorithm implicitly defines the space of all hypotheses that the program can ever represent and therefore can ever learn.

General-to-Specific Ordering of Hypotheses

Consider two hypotheses, **- h1 = (Sunny, ?, ?, Strong, ?, ?) and - h2 = (Sunny, ?, ?, ?, ?, ?)**

By looking at the 2 hypothesis representations, the hypothesis h2 has fewer constraints on attributes than hypothesis h1. The sets of instances that are classified positive by hl will be less than sets of instances that are classified positive by h2 as h2 imposes fewer constraints on the instance.

In other words, we can also say that, any instance classified positive by hl will also be classified positive by h2.

Therefore, we say that h2 is more general than hl. In reverse, we can also say that, h1 is more specific than h2.

Lets formalize this concept. Hypothesis h1 and h2 classifies the instance x as positive can written as h1(x) = 1 and h2(x) = 1. Now h2 is more general than h1, so it can be written as if h1(x) = 1 implies h2(x) = 1. In symbols ..

$$h_2 \geq_g h_1$$

Now lets describe a definition for **more-general-than-or-equal-to —**

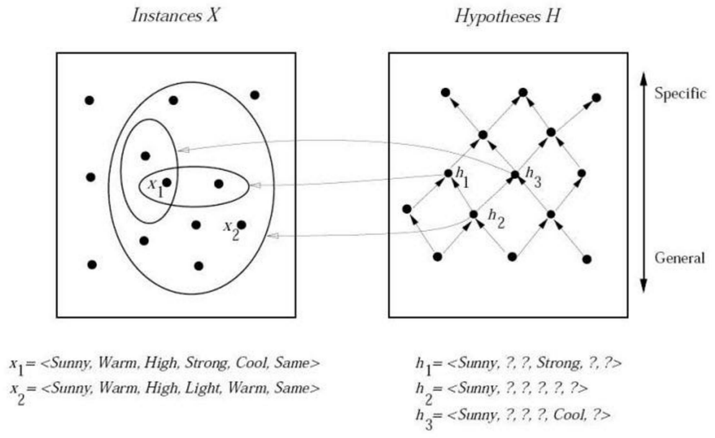
Definition: Let h_j and h_k be Boolean-valued functions defined over X. Then h_j is **more general-than-or-equal-to** h_k (written $h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[h_k(x) = 1 \rightarrow h_j(x) = 1]$$

In similar way, **more-specific-than**, also be defined as below.

The h_j is more-general-than h_k ($h_j > h_k$) if and only if $h_j \geq_g h_k$ is true and $h_k \geq_g h_j$ is false. We also say h_k is **more-specific-than** h_j.

By now, we understood the kind of relationship could exist between 2 hypothesis like more-general-than-equal-to or more-general-than. Lets illustrate this relationships using the below figure.



- In the figure, the box on the left represents the set X of all instances, the box on the right the set H of all hypotheses.
- Each hypothesis corresponds to some subset of X, that is, the subset of instances that it classifies positive. In the figure, there are 2 instances x1 and x2, and 3 hypothesis h1, h2 and h2.
- h1 classifies — x1, h2 classifies — x1 and x2, and h3 classifies — x1. This indicates, h2 is more-general-than h1 and h3.
- The arrows connecting hypotheses represent the more-general-than relation, with the arrow pointing toward the less general hypothesis.
- Note the subset of instances characterized by h2 subsumes the subset characterized by hl , hence h2 is more-general–than h1

Many learning algorithms for concept learning organize the search through the hypothesis space by relying on a general-to-specific ordering of hypotheses. By taking advantage of this naturally occurring structure over the hypothesis space, we can design learning algorithms that exhaustively search even infinite hypothesis spaces without explicitly enumerating every hypothesis.

Find-s: finding a maximally specific hypothesis

Now after learning the concept of **general-to-specific ordering of hypotheses**, Now its time to use this partial ordering to organize the search for a hypothesis, that is consistent with the observed training examples. One way is to begin with the most specific possible hypothesis in H, then generalize this hypothesis each time it fails to cover an observed positive training example.

FIND-S algorithm is used for this purpose. Here are the steps for find-s algorithm.

1.Initialize h to the most specific hypothesis in H.
2.For each positive training instance x
For each attribute constraint a_i in h
If the constraint a_i is satisfied by x
Then do nothing
Else replace a_i in h by the next more general constraint that is satisfied by x
3. Output hypothesis h

To illustrate this algorithm, assume the learner is given the sequence of training examples from the EnjoySport task

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

The first step of FIND-S is to initialize h to the most specific hypothesis in H **h — (Ø, Ø, Ø, Ø, Ø, Ø)**

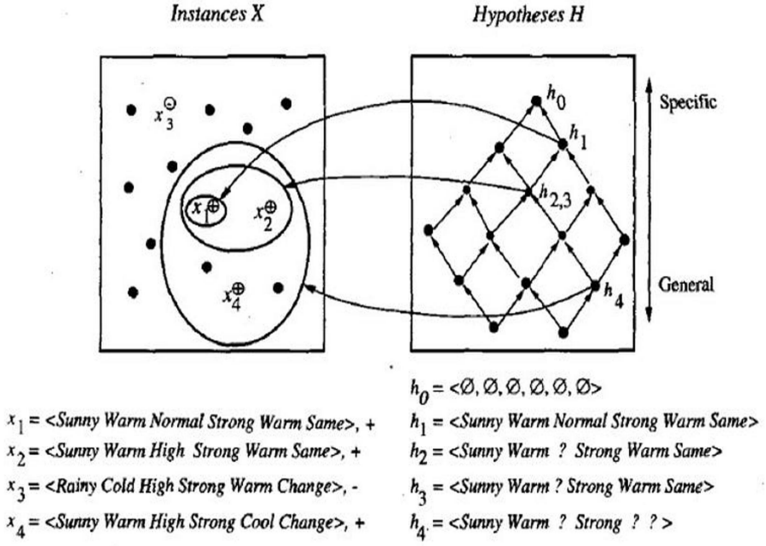
- First training example x1 = < Sunny, Warm, Normal, Strong ,Warm ,Same>, EnjoySport = +ve.

Observing the first training example, it is clear that hypothesis h is too specific.
None of the "Ø" constraints in h are satisfied by this example, so each is replaced by the next more general constraint that fits the example
h1 = < Sunny, Warm, Normal, Strong ,Warm, Same>.

- Consider the second training example x2 = < Sunny, Warm, High, Strong, Warm, Same>, EnjoySport = +ve.
  The second training example forces the algorithm to further generalize h, this time substituting a "?" in place of any attribute value in h that is not satisfied by the new example. Now **h2 =< Sunny, Warm, ?, Strong, Warm, Same>**
- Consider the third training example x3 =< Rainy, Cold, High, Strong, Warm, Change>,EnjoySport = — ve.
  The FIND-S algorithm simply ignores every negative example. So the hypothesis remain as before, so **h3 =< Sunny, Warm, ?, Strong, Warm, Same>**
- Consider the fourth training example x4 =<Sunny,Warm,High,Strong, Cool,Change>, EnjoySport =+ve.
  The fourth example leads to a further generalization of h as **h4 =< Sunny, Warm, ?, Strong, ?, ?>**

So the final hypothesis is **< Sunny, Warm, ?, Strong, ?, ?>**

The above example, can be illustrated with the below figure.



The search begins (ho) with the most specific hypothesis in H, then considers increasingly general hypotheses (hl through h4) as mandated by the training examples. The search moves from hypothesis to hypothesis, searching from the most specific to progressively more general hypotheses along one chain of the partial ordering. At each step, the hypothesis is generalized only as far as necessary to cover the new positive example. Therefore, at each stage the hypothesis is the most specific hypothesis consistent with the training examples observed up to this point.

The key property of the FIND-S algorithm —

- FIND-S is guaranteed to output the most specific hypothesis within H that is consistent with the positive training examples

- FIND-S algorithm's final hypothesis will also be consistent with the negative examples provided the correct target concept is contained in H, and provided the training examples are correct.

Unanswered questions by FIND-S

There are several questions still left unanswered, such as:

- **Has the learner converged to the correct target concept ?.**
  Although FIND-S will find a hypothesis consistent with the training data, it has no way to determine whether it has found the only hypothesis in H consistent with the data (i.e., the correct target concept), or whether there are many other consistent hypotheses as well.
- **Why prefer the most specific hypothesis ?.**
  In case there are multiple hypotheses consistent with the training examples, FIND-S will find the most specific. It is unclear whether we should prefer this hypothesis over, say, the most general, or some other hypothesis of intermediate generality.
- **Are the training examples consistent ?.**
  In most practical learning problems there is some chance that the training exampleswill contain at least some errors or noise. Such inconsistent sets of training examples can severely mislead FIND-S, given the fact that it ignores negative examples.
- **What if there are several maximally specific consistent hypotheses?.**
  There can be several maximally specific hypotheses consistent with the data. Find S finds only one.

Definition — Consistent

A hypothesis h is consistent with a set of training examples D if and only if h(x) = c(x) for each example (x, c(x)) in D.

$$Consistent\ (h, D) \equiv (\forall\ \langle x, c(x)\rangle \in D)\ h(x) = c(x))$$

Note the difference between definitions of consistent and satisfies

A training example x is said to satisfy hypothesis h when h(x) = 1, regardless of whether x is a positive or negative example of the target concept.

An example x is said to consistent with hypothesis h iff h(x) = c(x)

In the previous story, we discussed about FIND-S algorithm with an example. The set of training examples D are below.

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Also we found the hypothesis h = **< Sunny, Warm, ?, Strong, ?, ?>.** Now for each example (x, c(x)) in D, we will evaluate h(x) equals c(x).

1. (<Sunny, Warm, Normal, Strong, Warm, Same>,<yes>) → h(x)=c(x)
2. (<Sunny, Warm, High, Strong, Warm, Same>,<yes>) → h(x)=c(x)
3. (<Rainy, Cold, High, Strong, Warm, Change>,<No>) → h(x)=c(x)
4. (<Sunny, Warm, High, Strong, Cool, Change>,<yes>) → h(x)=c(x)

Now we can say, hypothesis h is consistent with a set of training examples D

Lets say, we have a hypothesis h1 = < ?, ?, ?, Strong, ?, ?>, is this hypothesis consistent with set of training example D ?
In case of training example (3), h(x) != c(x). So hypothesis h1 is not consistent with D.

Lets say, we have a hypothesis h2 = < ?, Warm, ?, Strong, ?, ?>, is this hypothesis consistent with set of training example D ?
All the training examples hold h(x) = c(x). So hypothesis h2 is consistent with D.

Definition — Version space
The version space, denoted VS_H,D with respect to hypothesis space H and training examples D, is the subset of hypotheses from H consistent with the training examples in D

$$VS_{H,D} \equiv \{h \in H \mid Consistent\ (h, D)\}$$

In above example, we have two hypothesis from H and they are consistent with D.
h=< Sunny, Warm, ?, Strong, ?, ?> and h2=< ?, Warm, ?, Strong, ?, ?>
So this set of hypothesis { h, h1} is called a Version Space.

The List-Then-Eliminate algorithm
One obvious way to represent the version space is simply to list all of its members. This leads to a simple learning algorithm, which we might call the List-Then-Eliminate algorithm. The algorithm is as follows :

1. ***VersionSpace*** *c a* list containing every hypothesis in H
2. For each training example, ***(x, c(x))***
   remove from ***VersionSpace*** any hypothesis ***h*** for which ***h(x) ≠ c(x)***
3. Output the list of hypotheses in ***VersionSpace***

The LIST-THEN-ELIMINATE algorithm first initializes the version space to contain all hypotheses in H and then eliminates any hypothesis found inconsistent with any training example.
In principle, List-Then-Eliminate algorithm can be applied whenever the hypothesis space H is finite. However, since it requires exhaustive enumeration of all hypotheses in practice it is not feasible.

Representation for Version Spaces :
we can represent the version space in terms of its most specific and most general members. For the above enjoysport training examples D, we can output the below list of hypothesis which are consistent with D. In other words, the below list of hypothesis is a version space.

h1	Sunny	?	?	?	?	?
h2	?	Warm	?	?	?	?
h3	Sunny	?	?	Strong	?	?
h4	Sunny	Warm	?	?	?	?
h5	?	Warm	?	Strong	?	?
h6	Sunny	Warm	?	Strong	?	?

In the list of hypothesis, there are two extremes representing general (h1 and h2) and specific (h6) hypothesis. Lets define these 2 extremes as general boundary G and specific boundary S.

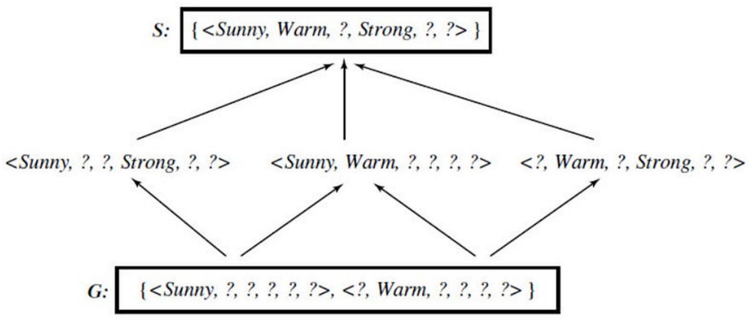
Definition — G
The **general boundary** G, with respect to hypothesis space H and training data D, is the set of maximally general members of H consistent with D.

Definition — S
The **specific boundary** S, with respect to hypothesis space H and training data D, is the set of minimally general (i.e., maximally specific) members of H consistent with D.

Version Space representation theorem —
Let X be an arbitrary set of instances and Let H be a set of Boolean-valued hypotheses defined over X. Let c: X →{O, 1} be an arbitrary target concept defined over X, and let D be an arbitrary set of training examples {(x, c(x))). For all X, H, c, and D such that S and G are well defined,

$$VS_{H,D}=\{h \in H \mid (\exists s \in S)\ (\exists g \in G)\ (g \geq_g h \geq_g s)\}$$

The below figure shows the version space for enjoysport concept learning including both general and specific boundary sets.



From Version Space representation theorem, there exist a hypothesis s belongs to specific boundary set S, and there exist a hypothesis g belongs to general boundary set G, a hypothesis h belongs hypothesis space H holding the relationship "g is **more-general-than-or-equal-to** h and h is **more-general-than-or-equal-to** s" forms a version space.

Candidate Elimination algorithm

- The Candidate-Elimination algorithm computes the version space containing all hypotheses from H that are consistent with an observed sequence of training examples.
- It begins by initializing the version space to the set of all hypotheses in H; that is, by initializing the G boundary set to contain the most general hypothesis in H as
  G0 ← { <?, ?, ?, ?, ?, ?> }
  and initializing the S boundary set to contain the most specific hypothesis as
  S0 ← { <0, 0, 0, 0, 0, 0> }
- These two boundary sets delimit the entire hypothesis space, because every other hypothesis in H is both more general than S0 and more specific than G0.
- As each training example is considered, the S and G boundary sets are generalized and specialized, respectively, to eliminate from the version space any hypotheses found inconsistent with the new training example.
- After all examples have been processed, the computed version space contains all the hypotheses consistent with these examples and only these hypotheses.
  Lets look into the algorithm steps :

Initialize G to the set of maximally general hypotheses in H

Initialize S to the set of maximally specific hypotheses in H

For each training example d, do

- If d is a positive example
  - Remove from G any hypothesis inconsistent with d
  - For each hypothesis s in S that is not consistent with d
    - Remove s from S
    - Add to S all minimal generalizations h of s such that
      - h is consistent with d, and some member of G is more general than h
    - Remove from S any hypothesis that is more general than another hypothesis in S

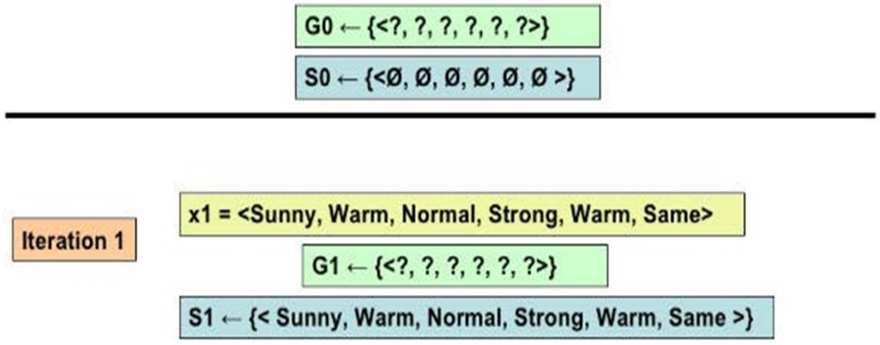
- If d is a negative example
  - Remove from S any hypothesis inconsistent with d
  - For each hypothesis g in G that is not consistent with d
    - Remove g from G
    - Add to G all minimal specializations h of g such that
      - h is consistent with d, and some member of S is more specific than h
    - Remove from G any hypothesis that is less general than another hypothesis in G

Candidate elimination algorithm with an example

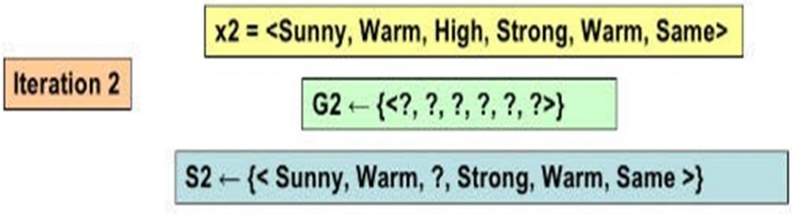
Here are the training examples D

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

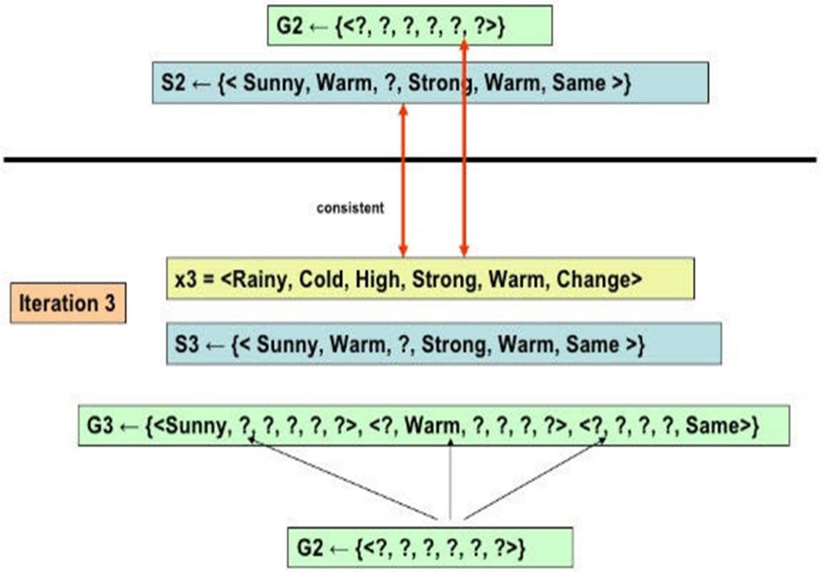
- CANDIDATE-ELIMINTION algorithm begins by initializing the version space to the set of all hypotheses in H.
- Initializing the G boundary set to contain the most general hypothesis in H G0 = <?, ?, ?, ?, ?, ?>
- Initializing the S boundary set to contain the most specific (least general) hypothesis S0 <0,0,0,0,0,0>
- First training example — its a positive example and when its presented to the CANDIDATE-ELIMINTION algorithm, it checks the S boundary and finds that it is overly specific and it fails to cover the positive example. The boundary is therefore revised by moving it to the least more general hypothesis that covers this new example
- No update of the G boundary is needed in response to this training example because G0 correctly covers this example



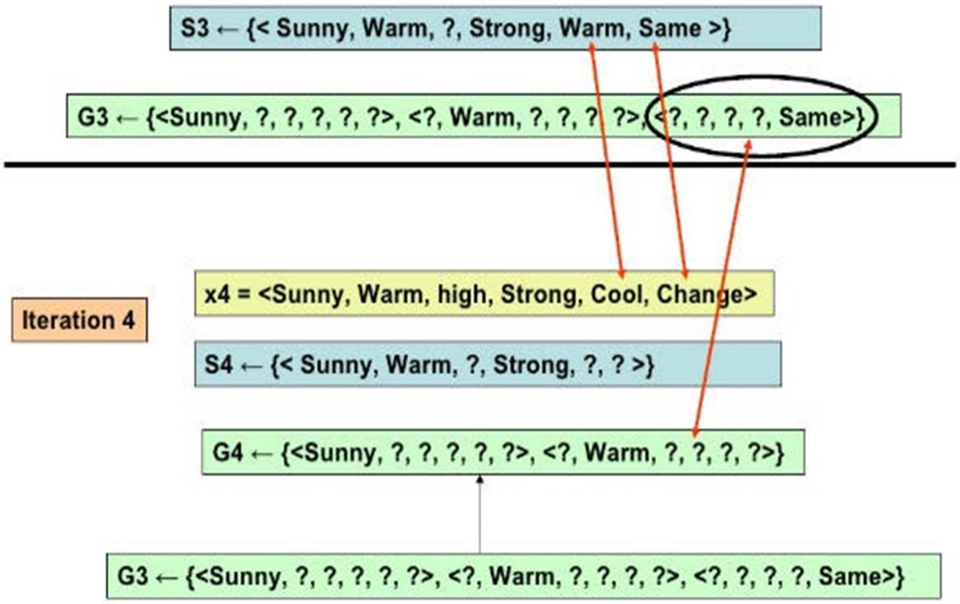
- When the second training example is presented, it has a similar effect of generalizing S further to S2, leaving G again unchanged i.e., G2 = G1 = G0



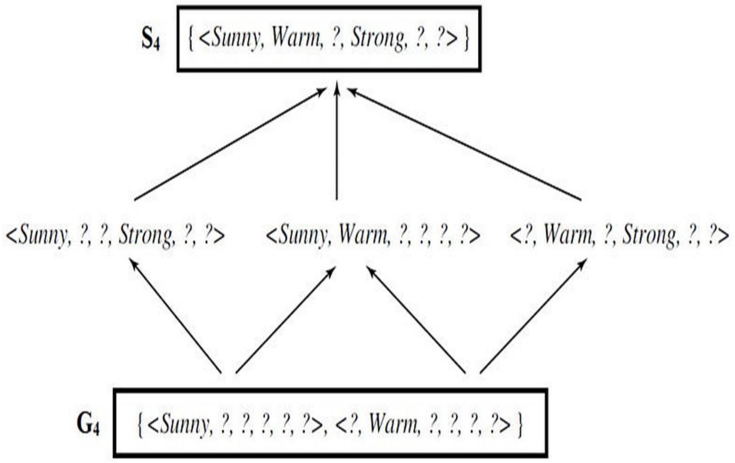
- Now the third training example — its a negative example and when its presented to the CANDIDATE-ELIMINTION algorithm, it reveals that the G boundary of the version space is overly general, that is, the hypothesis in G incorrectly predicts that this new example is a positive example. The hypothesis in the G boundary must therefore be specialized until it correctly classifies this new negative example.



- Given that there are six attributes that could be specified to specialize G2, why are there only three new hypotheses in G3 ?
  For example, the hypothesis h = (?, ?, Normal, ?, ?, ?) is a minimal specialization of G2 that correctly labels the new example as a negative example, but it is not included in G3.
  The reason is, this hypothesis is excluded as it is inconsistent with the previously encountered positive examples.
- Now the fourth training example — its a positive example and when its presented to the CANDIDATE-ELIMINTION algorithm, it further generalizes the S boundary of the version space. It also results in removing one member of the G boundary, because this member fails to cover the new positive example



- After processing these four examples, the boundary sets S4 and G4 delimit the version space of all hypotheses consistent with the set of incrementally observed training examples. The entire version space, including those hypotheses bounded by S4 and G4.



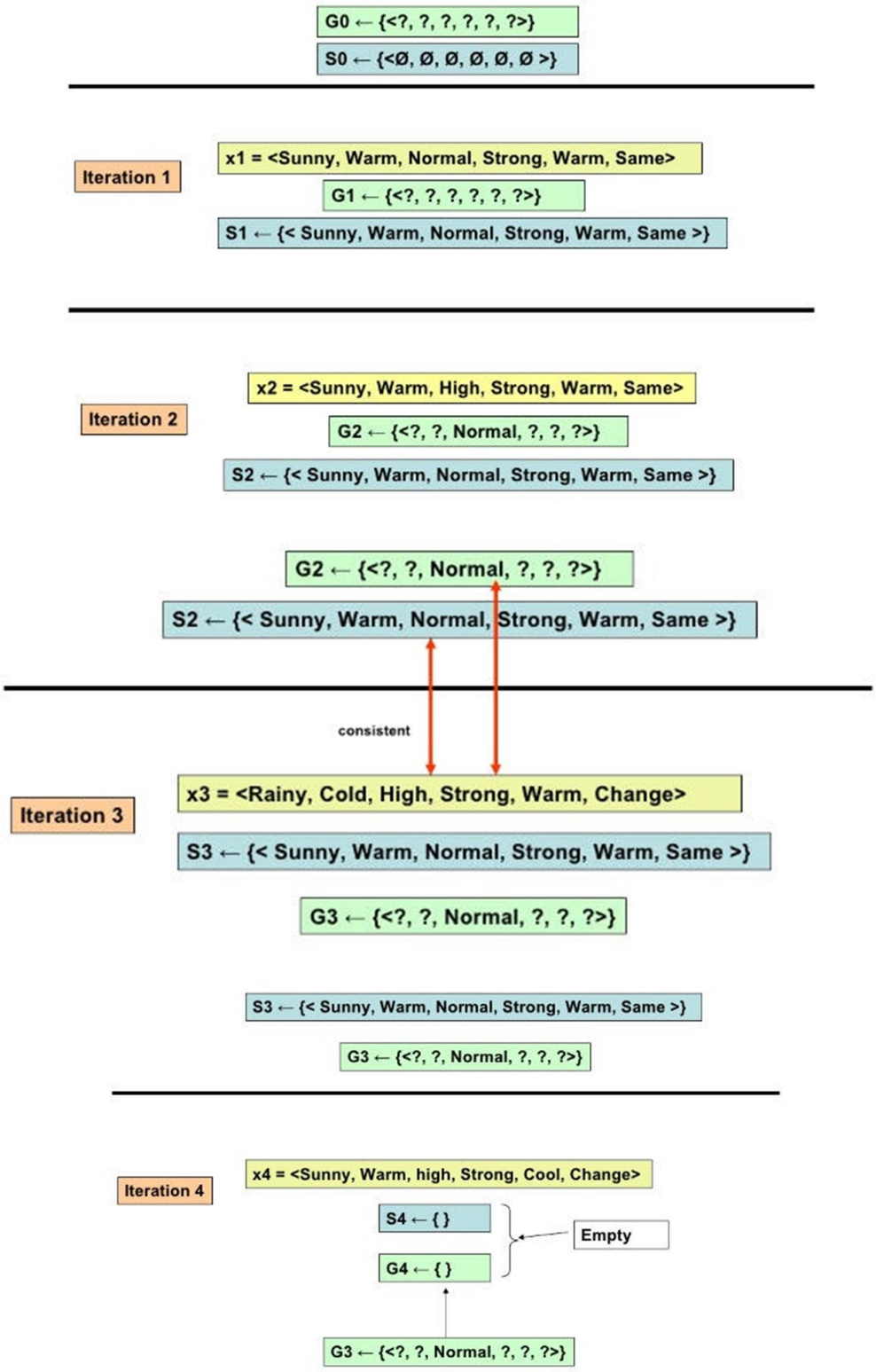
- This learned version space is independent of the sequence in which the training examples are presented (because in the end it contains all hypotheses consistent with the set of examples).
- As further training data is encountered, the S and G boundaries will move monotonically closer to each other, delimiting a smaller and smaller version space of candidate hypotheses.

Remarks on Version Space and Candidate elimination algorithm
Will the Candidate elimination algorithm Converge to the Correct Hypothesis ?
The version space learned by the Candidate elimination algorithm will converge toward the hypothesis that correctly describes the target concept, provided
(1) there are no errors in the training examples, and
(2) there is some hypothesis in H that correctly describes the target concept.

What will happen if the training data contains errors ?.
Suppose, for example, that the second training example above is incorrectly presented as a negative example instead of a positive example.

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	No
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Let's run the candidate elimination algorithm on this data and see the result.



After processing all the training examples, the algorithm removes the correct target concept from the version space.

The S and G boundary sets eventually converge to an empty version space if sufficient

additional training data is available. Such an empty version space indicates that there is no hypothesis in H consistent with all observed training examples.

A similar problem will appear when the training examples are correct, but the target concept cannot be described in the hypothesis representation. For example, if the target concept is a disjunction of feature attributes and the hypothesis space supports only conjunctive descriptions.

Every machine learning algorithm with any ability to generalize beyond the training data that it sees has, by definition, some type of inductive bias. That is, there is some **fundamental assumption or set of assumptions** that the learner makes about the target function that enables it to generalize beyond the training data.

Below is a chart that shows the inductive biases for various machine learning algorithms:

Algorithm	Inductive Bias
Rote-Learner	None
Candidate-Elimination	The target concept c is contained in the hypothesis space H.
Find-S	The target concept can be described in its hypothesis space. All instances are negative instances unless demonstrated otherwise.
Linear Regression	The relationship between the attributes x and the output y is linear. The goal is to minimize the sum of squared errors.
Decision Trees	Shorter trees are preferred over longer trees. Trees that place high information gain attributes close to the root are preferred over those that do not.
Single-Unit Perceptron	Each input votes independently toward the final classification (interactions between inputs are not possible).
Neural Networks with Backpropagation	Smooth interpolation between data points.
K-Nearest Neighbors	The classification of an instance x will be most similar to the classification of other instances that are nearby in Euclidean distance.
Support Vector Machines	Distinct classes tend to be separated by wide margins.
Naive Bayes	Each input depends only on the output class or label; the inputs are independent from each other.

we saw that, the candidate elimination algorithm converge towards the true target concept provided it is given accurate training examples and provided its initial hypothesis space contains the target concept.

- What if the target concept is not contained in the hypothesis space?
- Can we avoid this difficulty by using a hypothesis space that includes every possible hypothesis ?
- How does the size of this hypothesis space influence the ability of the algorithm to generalize to unobserved instances ?
- How does the size of the hypothesis space influence the number of training examples that must be observed ?

These are fundamental questions for inductive inference in general. Here we examine them in the context of the candidate elimination algorithm.

A Biased Hypothesis Space

We defined consistent as any hypothesis h is consistent with a set of training examples D if and only if h(x) = c(x) for each example (x, c(x)) in D.

Suppose the target concept c(x) is not contained in the hypothesis space H, then none of the hypothesis of H will be consistent with a set of training examples D. In that case, solution would be to enrich the hypothesis space to include every possible hypothesis. Consider the EnjoySport example in which the hypothesis space is restricted to include only conjunctions of attribute values.

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Cool	Change	Yes
2	Cloudy	Warm	Normal	Strong	Cool	Change	Yes
3	Rainy	Warm	Normal	Strong	Cool	Change	No

Because of this restriction, the hypothesis space is unable to represent even simple disjunctive target concepts such as "Sky = Sunny or Sky = Cloudy".

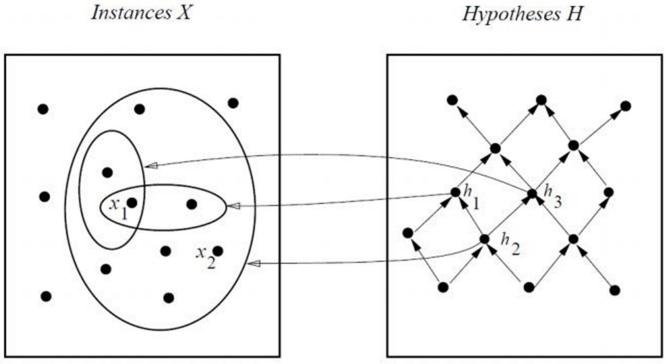
In the above three training examples, the target concept is "Sky = Sunny or Sky = Cloudy". Now we will check whether the candidate elimination algorithm will learn the concept.

After the second iteration, the algorithm will produce G2 = <?,?,?,?,?,?> and S2 = <? Warm Normal Strong Cool Change>.

This hypothesis, although it is the maximally specific hypothesis from H that is consistent with the first two examples, is already overly general: it incorrectly covers the third (negative) training example. The problem is that we have biased the learner to consider only conjunctive hypotheses. In this case we require a more expressive hypothesis space.

An Unbiased Learner

In the following figure, the box on the left represents the set X of all instances, the box on the right the set H of all hypotheses. Each hypothesis corresponds to some subset of X.



Suppose if we have 3 instances then we can have pow(2,3) = 8 subsets. Each subset corresponds to one hypothesis in hypothesis space. Each hypothesis will learn a concept that is represented by the subset of the instances. By having such a hypothesis space will represent every teachable concept that is representing every possible subset of the instances X.

Note — out of 8 hypothesis, only 1 hypothesis is a conjunctive and rest 7 hypothesis are disjunctions, conjunctions, and negations combinations.

In the EnjoySport learning task the size of the instance space X of days described by the six attributes is (3.2.2.2.2.2 =) 96 instances. In total, we can have pow(2,96) subsets from the 96 district instances.

The solution to the problem of assuring that the target concept is in the hypothesis space H is to provide a hypothesis space capable of representing every teachable concept that is representing every possible subset of the instances X.
The set of all subsets of a set X is called the power set of X.
Note, the conjunctive hypothesis space is able to represent only (4.3.3.3.3.3 =)973 of these — a biased hypothesis space indeed.

Let us reformulate the Enjoysport learning task in an unbiased way by defining a new hypothesis space H' that can represent every subset of instances; that is, let H' correspond to the power set of X. One way to define such an H' is to allow arbitrary disjunctions, conjunctions, and negations of our earlier hypotheses.

For instance, the target concept "Sky = Sunny or Sky = Cloudy" could then be described as (Sunny, ?, ?, ?, ?, ?) v (Cloudy, ?, ?, ?, ?, ?).

However, while this hypothesis space eliminates any problems of expressibility, it unfortunately raises a new, equally difficult problem: our concept learning algorithm is now completely unable to generalize beyond the observed examples!.
For example, Let Positive example be (x1, x2, x3) and negative example be (x 4, x5). Now the S and G boundary will be
S:{(x1 ∨ x 2 ∨ x 2)}, G:{ ¬ (x 4 ∨ x 5)}
S boundary will always be simply the disjunction of the observed positive examples, while the G boundary will always be the negated disjunction of the observed negative examples. The only examples that will be classified by S and G are the observed training examples themselves. In order to converge to a single, final target concept, we will have to present every single instance in X as a training example!

The Futility of Bias-Free Learning
The fundamental property of inductive inference —
***"a learner that makes no a priori assumptions regarding the identity of the target concept has no rational basis for classifying any unseen* instances"**
In fact, the only reason that the candidate elimination algorithm was able to generalize beyond the observed training examples in our original formulation of the EnjoySport task is that it was biased by the implicit assumption that the target concept could be represented by a conjunction of attribute values. In cases where this assumption is correct (and the training examples are error-free), its classification of new instances will also be correct. If this assumption is incorrect, however, it is certain that the candidate elimination algorithm will miss-classify at least some instances from X.

Let us define this notion of inductive bias more precisely.

- Consider a concept learning algorithm L for the set of instances X. Let c be an arbitrary concept defined over X, and let Dc = {<x,c(x)>} be an arbitrary set of training examples of c.
- After training, L is asked to classify a new instance xi. Let L(xi, Dc) denote the classification (e.g., positive or negative) that L assigns to xi after learning from the training data Dc.
- We can describe this inductive inference step performed by L as follows
- $(D_c \wedge x_i) \succ L(x_i \wedge D_c)$
- Where the notation y > z indicates that z is inductively inferred from y. For example, if we take L to be the candidate elimination algorithm, Dc, to be the training data as below, and xi to be the fist instance from test data as below, then the inductive inference performed in this case concludes that L(xi, Dc) =(EnjoySport = yes).

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Training Data D

Instance	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
A	Sunny	Warm	Normal	Strong	Cool	Change	?
B	Rainy	Cold	Normal	Light	Warm	Same	?
C	Sunny	Warm	Normal	Light	Warm	Same	?
D	Sunny	Cold	Normal	Strong	Warm	Same	?

Test Data

Definition:

Consider a concept learning algorithm L for the set of instances X. Let c be an arbitrary concept defined over X, and let Dc = {⟨x, c(x) ⟩} be an arbitrary set of training examples of c. Let L(xi, Dc) denote the classification assigned to the instance xi by L after training on the data Dc. The inductive bias of L is any minimal set of assertions B such that for any target concept c and corresponding training examples Dc

$$(\forall x_i \in X)\ [(B \wedge D_c \wedge x_i) \vdash L(x_i, D_c)]$$

y | — z indicates that z follows deductively from y (i.e., that z is provable from y).

Inductive Bias for Candidate Elimination

- Assume a training set Dc. The candidate elimination algorithm computes the version space VS_HD.
- Classify the new instance xi by a vote among hypotheses in this version space. Here let us assume that it will output a classification for xi only if this vote among version space hypotheses is unanimously positive or negative and that it will not output a classification otherwise.
- **Conjecture**: B = { c ∈ H } is the inductive bias ('the underlying target concept c is in H')
- From c ∈ H it follows that c is a member of the version space.

- L (xi, Dc) = k implies that all members of VS_HD, including c, vote for class k (unanimous voting). Therefore: c(xi) = k = L(xi, Dc).
- This means, that the output of the learner L(xi, Dc) can be logically deduced from B ∧ Dc ∧ xi
- → The inductive bias of the Candidate Elimination Algorithm is: **"c is in H"**

Inductive Systems and Equivalent Deductive Systems

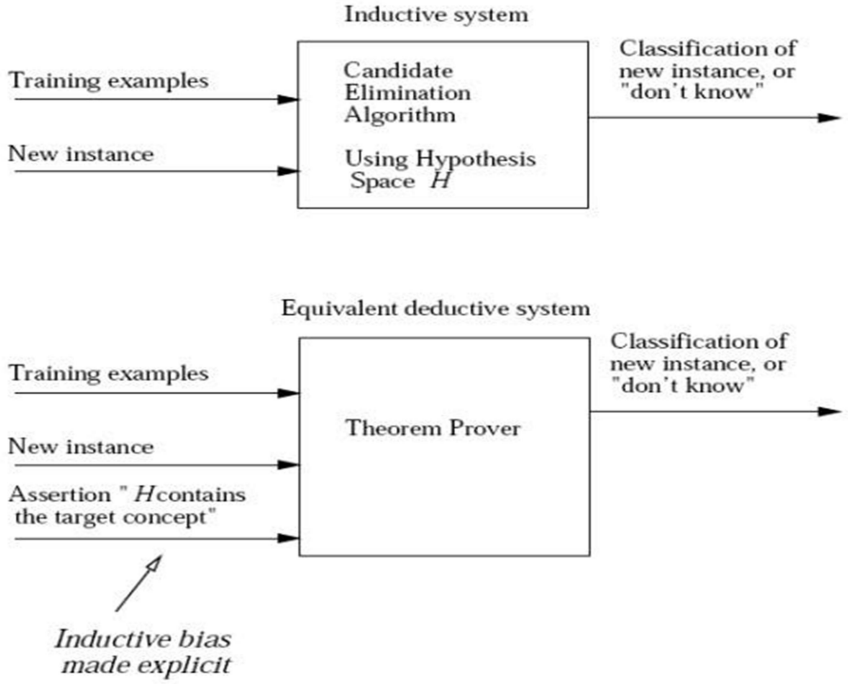
We saw two terms — inductive and deductive. But what is the difference between these two — inductive and deductive logic ?

Deductive logic uses given information, premises or accepted general rules to reach a proven conclusion. So that is the reason, we called that the learner L(xi, Dc) can be logically deduced from B ∧ Dc ∧ xi.

On the other hand, inductive logic or reasoning involves making generalizations based upon behavior observed in specific cases. So that is the reason, we called that the learner L(xi,Dc) inductively inferred from (Dc ∧ xi). It depends on choice of the learning algorithm used and what assumption it makes.

The below figure explains

- Modelling inductive systems by equivalent deductive systems.
- The input-output behavior of the candidate elimination algorithm using a hypothesis space H is identical to that of a deductive theorem prover utilizing the assertion "H contains the target concept." This assertion is therefore called the inductive bias of the candidate elimination algorithm.
- Characterizing inductive systems by their inductive bias allows modelling them by their equivalent deductive systems. This provides a way to compare inductive systems according to their policies for generalizing beyond the observed training data.



One advantage of viewing inductive inference systems in terms of their inductive bias is that it provides a non procedural means of characterizing their policy for generalizing beyond the observed data. A second advantage is that it allows comparison of different learners according to the strength of the inductive bias they employ.

The following three learning algorithms are listed from weakest to strongest bias.
1.Rote-learning : storing each observed training example in memory. If the instance is found in memory, the store classification is returned.
Inductive bias : nothing — Weakest bias
2.Candidate-Elimination algorithm : new instances are classified only in the case where all members of the current version space agree in the classification.
Inductive bias : Target concept can be represented in its hypothesis space
3. Find-S : find the most specific hypothesis consistent with the training examples. It then uses this hypothesis to classify all subsequent instances.
Inductive bias : Target concept can be represented in its hypothesis space + All instances are negative instances unless the opposite is entailed by its other knowledge — Strongest bias
More strongly biased methods make more inductive leaps, classifying a greater proportion of unseen instances!!

Probably approximately correct learning

In computer science, computational learning theory (or just learning theory) is a subfield of artificial intelligence devoted to studying the design and analysis of machine learning algorithms. In computational learning theory, probably approximately correct learning (PAC learning) is a framework for mathematical analysis of machine learning algorithms. It was proposed in 1984 by Leslie Valiant.
In this framework, the learner (that is, the algorithm) receives samples and must select a hypothesis from a certain class of hypotheses. The goal is that, with high probability (the "probably" part), the selected hypothesis will have low generalization error (the "approximately correct" part). In this section we first give an informal definition of PAC-learnability. After introducing a few more notions, we give a more formal, mathematically oriented, definition of PAC-learnability.
PAC-learnability
To define PAC-learnability we require some specific terminology and related notations.

- Let X be a set called the instance space which may be finite or infinite. For example, X may be the set of all points in a plane.
- A concept class C for X is a family of functions c : X → {0; 1}. A member of C is called a concept. A concept can also be thought of as a subset of X. If C is a subset of X, it defines a unique function μ_c : X → {0; 1} as follows:

$$\mu_C(x) = \begin{cases} 1 & \text{if } x \in C \\ 0 & \text{otherwise} \end{cases}$$

- A hypothesis h is also a function h : X → {0; 1}. So, as in the case of concepts, a hypothesis can also be thought of as a subset of X. H will denote a set of hypotheses.
- We assume that F is an arbitrary, but fixed, probability distribution over X.
- Training examples are obtained by taking random samples from X. We assume that the samples are randomly generated from X according to the probability distribution F.

Definition (informal)

Let X be an instance space, C a concept class for X, h a hypothesis in C and F an arbitrary, but fixed, probability distribution. The concept class C is said to be PAC-learnable if there is an algorithm A which, for samples drawn with any probability distribution F and any concept c Є C, will with high probability produce a hypothesis h Є C whose error is small.

Examples

To illustrate the definition of PAC-learnability, let us consider some concrete examples.

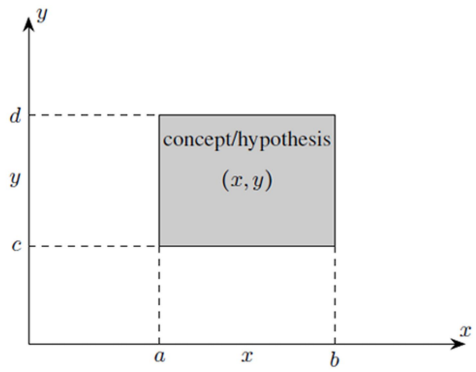


Figure : An axis-aligned rectangle in the Euclidean plane

- Let the instance space be the set X of all points in the Euclidean plane. Each point is represented by its coordinates (x; y). So, the dimension or length of the instances is 2.
- Let the concept class C be the set of all "axis-aligned rectangles" in the plane; that is, the set of all rectangles whose sides are parallel to the coordinate axes in the plane (see Figure).
- Since an axis-aligned rectangle can be defined by a set of inequalities of the following form having four parameters

$$a \le x \le b, \quad c \le y \le d$$

the size of a concept is 4.
- We take the set H of all hypotheses to be equal to the set C of concepts, H = C.

Given a set of sample points labeled positive or negative, let L be the algorithm which outputs the hypothesis defined by the axis-aligned rectangle which gives the tightest fit to the positive examples (that is, that rectangle with the smallest area that includes all of the positive examples and none of the negative examples) (see Figure bleow).

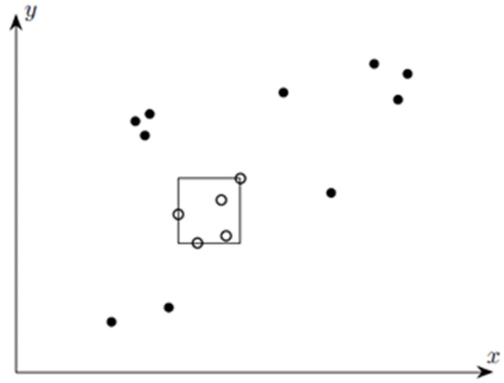


Figure : Axis-aligned rectangle which gives the tightest fit to the positive examples

It can be shown that, in the notations introduced above, the concept class C is PAC-learnable by the algorithm L using the hypothesis space H of all axis-aligned rectangles.

Applications of machine learning

Application of machine learning methods to large databases is called data mining. In data mining, a large volume of data is processed to construct a simple model with valuable use, for example, having high predictive accuracy.

The following is a list of some of the typical applications of machine learning.

- In retail business, machine learning is used to study consumer behaviour.
- In finance, banks analyze their past data to build models to use in credit applications, fraud detection, and the stock market.
- In manufacturing, learning models are used for optimization, control, and troubleshooting.
- In medicine, learning programs are used for medical diagnosis.
- In telecommunications, call patterns are analyzed for network optimization and maximizing the quality of service.
- In science, large amounts of data in physics, astronomy, and biology can only be analyzed fast enough by computers. The World Wide Web is huge; it is constantly growing and searching for relevant information cannot be done manually.
- In artificial intelligence, it is used to teach a system to learn and adapt to changes so that the system designer need not foresee and provide solutions for all possible situations.
- It is used to find solutions to many problems in vision, speech recognition, and robotics.
- Machine learning methods are applied in the design of computer-controlled vehicles to steer correctly when driving on a variety of roads.
- Machine learning methods have been used to develop programmes for playing games such as chess, backgammon and Go.

Real life examples

i) Optical character recognition

Optical character recognition problem, which is the problem of recognizing character codes from their images, is an example of classification problem. This is an example where there are multiple classes, as many as there are characters we would like to recognize. Especially interesting is the case when the characters are handwritten. People have different handwriting styles; characters may be written small or large, slanted, with a pen or pencil, and there are many possible images corresponding to the same character.

ii) Face recognition

In the case of face recognition, the input is an image, the classes are people to be recognized,and the learning program should learn to associate the face images to identities. This problem is more difficult than optical character recognition because there are more classes, input image is larger, and a face is three-dimensional and differences in pose and lighting cause significant changes in the image.

iii) Speech recognition

In speech recognition, the input is acoustic and the classes are words that can be uttered.

iv) Medical diagnosis

In medical diagnosis, the inputs are the relevant information we have about the patient and the classes are the illnesses. The inputs contain the patient's age, gender, past medical history, and current symptoms. Some tests may not have been applied to the patient, and thus these inputs would be missing.

v) Knowledge extraction

Classification rules can also be used for knowledge extraction. The rule is a simple model that explains the data, and looking at this model we have an explanation about the process underlying the data.

vi) Compression

Classification rules can be used for compression. By fitting a rule to the data, we get an explanation that is simpler than the data, requiring less memory to store and less computation to process.

Here are some further examples of classification problems.

(a) An emergency room in a hospital measures 17 variables like blood pressure, age, etc.of newly admitted patients. A decision has to be made whether to put the patient in an ICU. Due to the high cost of ICU, only patients who may survive a month or more are given higher priority. Such patients are labeled as "low-risk patients" and others are labeled "high-risk patients". The problem is to device a rule to classify a patient as a "low-risk patient" or a "high-risk patient".

(b) A credit card company receives hundreds of thousands of applications for new cards.

The applications contain information regarding several attributes like annual salary, age, etc. The problem is to devise a rule to classify the applicants to those who are credit-worthy, who are not credit-worthy or to those who require further analysis.

(c) Astronomers have been cataloguing distant objects in the sky using digital images created using special devices. The objects are to be labeled as star, galaxy, nebula, etc.The data is highly noisy and are very faint. The problem is to device a rule using which a distant object can be correctly labeled.